

Orbit Wars — RL Math, v6 implementation (*diff* against `rl_math.tex`)

How to read this. This document is a *delta* on top of `rl_math.tex`: it restates only what the **v6** notebook (`notebooks/setup1_v6_ppo.ipynb`) changes, and for everything unchanged it points back by section. `rl_math.tex` describes the *ungated* Dirichlet target actor and the four-channel event reward; v6 keeps the same trunk, simulator, and Elo league but changes (i) the action distribution to a **gated** allocation with a sharpened greedy, (ii) the reward to **potential-based shaping**, (iii) several PPO/GAE knobs and adds **PopArt** value normalization, (iv) the destination head to a **bilinear pointer**, and (v) the data path to be **on-GPU** / **bf16** / **sync-free** for an A100. Notation, symbols, and any equation cited by its `rl_math.tex` label (e.g. `eq:return`) live in `rl_math.tex`.

One-line map of what changed.

<code>rl_math.tex</code> section	status in v6	where
§ Problem setup (<code>sec:problem</code>)	unchanged	§1
§ Lead-target aiming (<code>sec:lead</code>)	unchanged	§1
§ Observation encoding (<code>sec:obs</code>)	unchanged	§1
§ Policy network (<code>sec:policy</code>)	trunk same; heads changed	§2
§ Action distribution (<code>sec:dist</code>)	replaced (gated)	§3
§ Reward and return (<code>sec:reward</code>)	replaced (shaping)	§4
§ Curriculum / league (<code>sec:curriculum, sec:league</code>)	mostly same	§5
§ KL / trust region (<code>sec:kldiv</code>)	unchanged	§1
§ PPO + GAE appendix (<code>sec:ppo</code>)	knobs changed; +PopArt	§6
§ GPU simulator (<code>sec:gpuenv</code>)	dynamics same; data path	§7
§ Search appendix (<code>sec:search</code>)	unchanged (not built)	§1

1 Unchanged — see `rl_math.tex`

The following are **identical** in v6; consult `rl_math.tex` directly:

- Problem setup, MDP, two-player zero-sum scoring, the engine launch model: see `rl_math.tex` § “**Problem setup**” (`sec:problem`).
- Deterministic orbital-intercept aiming (`LEAD_TARGET`): see `rl_math.tex` § “**Lead-target aiming**” (`sec:lead`) (used verbatim by the v6 decode).
- Observation encoding — body rows, the incoming-fleet threat block, globals, all feature scalings: see `rl_math.tex` § “**Observation encoding**” (`sec:obs`) and see `rl_math.tex` § “**threat block**” (`sec:threat`). ($F=101$, $G=10$ unchanged.)
- KL estimators (k_1, k_2, k_3), `approx_kl`, `clipfrac`, the clipped surrogate definition: see `rl_math.tex` § “**KL divergence and the PPO trust region**” (`sec:kldiv`).

- The Elo/PFSP league mechanics, scripted anchors, Elo back-solve recalibration, matchmaking weight: see `rl_math.tex` § “Self-play league” (`sec:league`) and see `rl_math.tex` § “interventions” (`sec:interventions`) (only the two items in §5 differ).
- Batched GPU simulator dynamics and sim-to-real fidelity: see `rl_math.tex` § “Batched GPU simulator” (`sec:gpuenv`) (only the *data path* differs, §7).
- Search-based improvement (AlphaZero/MuZero): see `rl_math.tex` § “search-based policy improvement” (`sec:search`). **Not built in v6** (no MCTS); the policy net is the deployed agent.

2 Policy network: heads

The **trunk is unchanged** — input projection, one (or N_{tx}) masked cross-planet self-attention block(s), GLU/residual-MLP stack, final LayerNorm, separate globals embedding g' : see `rl_math.tex` § “Policy network” (`sec:policy`), including the trunk-vs-transformer sizing of see `rl_math.tex` § “Recommended hyperparameters” (`sec:trunkcfg`). v6 changes only the *heads* and one attention *kernel*.

(a) **Attention kernel (math-neutral).** The attention of Eq. (`sec:policy`) is computed with `F.scaled_dot_product_attention` (a fused flash/mem-efficient kernel) under a bf16 autocast; the dead-planet key mask is supplied as an additive -10^9 bias. Identical function, faster kernel.

(b) **Bilinear (pointer) destination head.** `rl_math.tex` uses a single `Linear( $d+d_g \rightarrow E$ )` for the `dest_head`. v6 replaces it (flag `DEST_HEAD="bilinear"`) with a pairwise pointer score over the source/destination tokens $h'_e = [h_e; g'] \in \mathbb{R}^{d+d_g}$:

$$\ell_{s,d} = \frac{(W_Q h'_s)^\top (W_K h'_d)}{\sqrt{d}}, \quad W_Q, W_K \in \mathbb{R}^{d \times (d+d_g)}. \quad (1)$$

The score for sending $s \rightarrow d$ now depends on *both* planets’ features (relative geometry, ownership, garrison, threat), instead of a fixed per-slot weight vector — the natural inductive bias for a relational allocation. *Calm init*: W_Q, W_K scaled by `INIT_DEST_SCALE` = 0.02 so $\ell \approx 0$ (near-uniform WHERE) at start; “hold by default” is enforced by the gate bias below, not by a diagonal logit bias.

(c) **Launch-gate head.** v6 adds `gate_head`: a `Linear( $d+d_g \rightarrow 1$ )` emitting a per-source fire logit η_s (§3). Bias initialized to `INIT_GATE_BIAS` = -1 (few planets fire at start).

(d) **Value head predicts a *normalized* value (PopArt).** The critic architecture is unchanged (masked-mean pool of trunk tokens $\oplus g' \rightarrow \text{VALUE_RES_BLOCKS}$ pre-LN residual blocks $\rightarrow d \rightarrow 1$; see `rl_math.tex` § “PPO + GAE” (`sec:ppo`)). In v6 its scalar output is the *normalized* value $\tilde{V}_\theta(s)$; the usable value is $V(s) = \sigma \tilde{V}_\theta(s) + \mu$ with the running (μ, σ) of §6. GAE and bootstrapping use the de-normalized V .

3 Action distribution: gated allocation

`rl_math.tex` § `sec:dist` makes each owned source row a Dirichlet over the destination simplex with the *self-diagonal as HOLD*, and evaluates greedily at the Dirichlet *mean*. v6 keeps the Dirichlet

for *where* but adds an explicit **Bernoulli launch gate** for *whether*, and replaces the greedy rule. The reachability mask $R_{s,j}$ (sun occlusion) and the owned-source factorization are exactly as in `sec:dist`.

Two factors per owned source s .

$$\textbf{WHERE:} \quad \alpha_{s,j} = \text{softplus}(\text{clip}(\ell_{s,j}, \pm \text{LOGIT_CLAMP})) \cdot \text{ALLOC_KAPPA} + \epsilon, \quad A[s, \cdot] \sim \text{Dirichlet}(\alpha_s), \quad (2)$$

$$\textbf{WHETHER:} \quad g_s = \sigma(\eta_s), \quad p_s = \text{clip}\left(\frac{g_s - \text{GATE_TRIM_LO}}{\text{GATE_TRIM_HI} - \text{GATE_TRIM_LO}}, 0, 1\right), \quad p_s \leftarrow \text{clip}(p_s, \text{GATE_EPS}, 1 - \text{GATE_EPS}), \quad (3)$$

$$f_s \sim \text{Bernoulli}(p_s) \in \{0, 1\}. \quad (4)$$

The trim $(\text{GATE_TRIM_LO}, \text{GATE_TRIM_HI}) = (0.2, 0.8)$ gives a dead-zone / saturation band so the gate can express “essentially never/always fire”; the final clip to $[\text{GATE_EPS}, 1 - \text{GATE_EPS}]$, $\text{GATE_EPS} = 0.02$, keeps a saturated gate *trainable* (nonzero gradient) during sampling. The sampled action is bundled as the $(B, E, E+1)$ tensor $a = [A \mid f]$.

Log-prob and entropy (the WHERE term is gated by firing).

$$\log \pi_\theta(a \mid s) = \sum_{s: \text{owned}} \left[\underbrace{f_s \log p_s + (1-f_s) \log(1-p_s)}_{\text{Bernoulli gate}} + f_s \log \text{Dirichlet}(A[s, \cdot] \mid \alpha_s) \right], \quad (5)$$

$$\mathcal{H}[\pi_\theta(\cdot \mid s)] = \sum_{s: \text{owned}} \left[\mathcal{H}_{\text{Bern}}(p_s) + p_s \mathcal{H}_{\text{Dir}}(\alpha_s) \right]. \quad (6)$$

The WHERE log-prob counts *only* when the planet fired ($f_s=1$): on a hold the destination row is unobserved, so it must not contribute to the score (otherwise a holding planet would be credited/penalized for an allocation it never executed). Symmetrically the WHERE entropy is weighted by the fire probability p_s .

Decode (see `rl_math.tex` § “Decode” (`sec:decode`) with the gate). A firing source dispatches its **full garrison** routed by its renormalized off-diagonal row; a holding source keeps everything:

$$\hat{A}[s, d] = \frac{A[s, d] (1 - \delta_{sd})}{\sum_{d' \neq s} A[s, d']}, \quad n_{s,d} = \lfloor \hat{A}[s, d] S_s f_s \rfloor \text{ for alive } d, \quad n_{s,d} \geq \text{MIN_LAUNCH_SHIPS}. \quad (7)$$

Aiming is the orbital intercept of `sec:lead`. Because the row is a simplex and $f_s \in \{0, 1\}$, total dispatch is 0 (hold) or exactly the garrison spread over destinations (fire); over-dispatch is structurally impossible, as in `sec:dist`.

Greedy / deploy — the v6 change. `rl_math.tex` deploys at the Dirichlet *mean*, which is concentration-blind: a moderately-preferred row (ℓ gap ~ 2) has mean mass $\approx \text{softplus}(2) / \sum \text{softplus} \approx 0.06$ on the preferred destination, so the full garrison spreads thin and *every* $\lfloor \cdot \rfloor$ floors below $\text{MIN_LAUNCH_SHIPS} \Rightarrow$ zero launches (a residual greedy-passive failure that the gate alone does not

fix). v6 deploys with a **temperature-sharpened, masked softmax** over the live, sun-reachable, *off-diagonal* destinations and a hard gate:

$$A^{\text{greedy}}[s, \cdot] = \text{softmax}\left(\tilde{\ell}_s/\tau\right), \quad \tilde{\ell}_{s,j} = \ell_{s,j} \text{ if } (j \neq s) \wedge m_j \wedge R_{s,j}, \text{ else } -\infty; \quad f_s^{\text{greedy}} = \mathbb{1}[p_s \geq 0.5] \quad (8)$$

with $\tau = \text{GREEDY_TAU} = 0.5$. This concentrates the garrison on the best valid destination (a real strike) instead of dust, and never wastes mass on dead/unreachable/self slots. *Verified*: in the moderate-preference regime the mean greedy fires 0 launches while the sharpened greedy fires one concentrated strike per firing source. Sampling is unchanged (the unmasked Dirichlet + Bernoulli; the decode and the natural sun-occlusion signal still apply).

4 Reward and return: potential-based shaping

`rl_math.tex` § `sec:reward` uses a four-channel *event* reward $R = O + C + D + P$ (outcome, capture, dispatch, production-milestone). Those dense channels are *farmable*: capture rewards border-flip churn, dispatch/milestone reward ship expenditure/hoarding — all maximizable while *losing*. v6 (flag `USE_POTENTIAL_SHAPING`, default on) **replaces** C, D, P with **potential-based shaping** (Ng, Harada & Russell, 1999), keeping the outcome O exactly as in `rl_math.tex` Eq. (`eq:return`). The legacy four-channel reward is retained verbatim behind `USE_POTENTIAL_SHAPING=False` ($\equiv$ `sec:reward`).

Potentials. With per-side total ships S^0, S^1 (planets + fleets), the ship-log $\ell(n) = \ln(1+n)/\ln 1000$, per-side owned production Π^0, Π^1 and total alive production Π_{tot} ,

$$\Phi_{\text{ship}}(s) = \ell(S^0) - \ell(S^1), \quad \Phi_{\text{prod}}(s) = \frac{\Pi^0 - \Pi^1}{\max(\Pi_{\text{tot}}, 1)}. \quad (9)$$

Both are zero-sum, bounded ($\Phi_{\text{ship}} \in [-1, 1]$, $\Phi_{\text{prod}} \in [-1, 1]$), and directly track the win condition (ship margin / board-control margin).

Shaping reward. The per-step reward is the shaping form

$$r_t = c_{\text{ship}}(\gamma \Phi_{\text{ship}}(s_{t+1}) - \Phi_{\text{ship}}(s_t)) + c_{\text{prod}}(\gamma \Phi_{\text{prod}}(s_{t+1}) - \Phi_{\text{prod}}(s_t)), \quad (10)$$

with $c_{\text{ship}} = \text{SHAPE_SHIP} = 50$, $c_{\text{prod}} = \text{SHAPE_PROD} = 50$, the *same* γ as GAE. As in `sec:reward` the outcome O is scattered onto the last active step, and both r_t and O are divided by $\sigma_R = \text{PPO_REWARD_SCALE} = 100$ before GAE.

Policy invariance (why this is un-hackable). For $F(s, s') = \gamma \Phi(s') - \Phi(s)$ the added discounted return telescopes,

$$\sum_{t \geq 0} \gamma^t F(s_t, s_{t+1}) = \gamma^T \Phi(s_T) - \Phi(s_0), \quad (11)$$

which is independent of the policy (only of the fixed start state and the terminal). Hence the optimal policy and ordering of policies are *unchanged* versus training on the sparse outcome O alone, while the gradient is now dense and win-aligned at every step. No reward-farming basin exists: a quantity that returns to its prior value (e.g. a flip-flopped border planet) contributes net zero. Empirically the outcome channel dominates the per-game return (e.g. $O \sim 450$ vs. shaping ~ 8 each), the intended regime.

Minor: comet-masked capture. In the legacy reward mode the capture channel now excludes comet slots (a comet that expires while ego-owned was previously miscounted as a lost planet). No effect in shaping mode.

5 Curriculum and league

Mechanics unchanged: see `rl_math.tex` § “Curriculum” (`sec:curriculum`) (the Elo league *is* the curriculum) and see `rl_math.tex` § “Self-play league” (`sec:league`). Two changes:

1. **Best-checkpoint selection = fixed-opponent gauntlet.** `rl_math.tex`/the prior notebook saved “best” on the per-iter win-rate *vs. whatever single PFSP opponent was drawn* (a weak net that drew `random` could be frozen as best). v6 selects on a deployment-faithful, opponent-independent gauntlet: greedy play vs. a fixed anchor set `{starter, intermediate, greedy}`, scored every `CKPT_EVERY` iters, $g = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \text{score}(\pi, a)$.
2. **Fingerprint probe on real states.** The behavioral fingerprint used by PFSP diversity is computed on a cached batch of *real* encoded game states (rolled out), not Gaussian-noise inputs, so the destination-distribution distance measures on-manifold behavior.

6 PPO + GAE: knobs, KL early-stop, PopArt

The actor-critic PPO + GAE objective, GAE recursion, and value loss are as in see `rl_math.tex` § “Appendix: PPO + GAE” (`sec:ppo`). v6 changes the following.

Horizon and sample reuse. $\gamma = \text{GAMMA} = 0.997$ (was 0.99) and $\lambda = \text{GAE_LAMBDA} = 0.98$ (was 0.95): a ~ 500 -step game with $\gamma=0.99$ discounts the terminal to $0.99^{500} \approx 7 \times 10^{-3}$ and confines GAE credit to the last $\sim 1/(1-\gamma\lambda)$ steps; raising both lets the (now win-aligned) signal reach mid-game actions. `UPDATE_EPOCHS= 3` (was 1) recovers real multi-epoch PPO sample reuse (one epoch is A2C-with-GAE).

KL early-stop (end-of-epoch). The prior code broke the update on the first *minibatch* whose k_3 KL exceeded `KL_TARGET` (a high-variance, order-dependent, subsample-biased stop). v6 (`KL_EARLYSTOP`) instead completes each epoch and stops *after* an epoch if its *mean* minibatch KL exceeds `KL_TARGET= 0.05`, a lower-variance trust-region control.

Entropy normalization (scale-correct). The entropy bonus is the mean *per owned source* entropy,

$$\bar{\mathcal{H}} = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \frac{\mathcal{H}_b}{\max(n_{\text{owned}}^b, 1)}, \quad n_{\text{owned}}^b = \sum_e \text{action_mask}_e^b, \quad (12)$$

replacing the prior $\bar{\mathcal{H}} = \mathcal{H}/(E \cdot \text{nap})$ with `nap=2` (a stale constant from the retired (dest, ϕ) actor whose magnitude drifted with how many planets the ego owned). The coefficient $c_{\mathcal{H}}$ is **annealed** `ENT_COEF_START = 0.02  $\rightarrow$  ENT_COEF_END = 0.002` over `ENT_DECAY_ITERS= 300` (explore $\rightarrow$ exploit), vs. a fixed `ent_coef`.

Terminal-correct GAE done. The episode done flag is now set at *any* terminal — both true elimination *and* the time cap — so $d_t = \mathbb{K}[\text{cap} \vee \text{death}]$. With the outcome scattered onto each env’s last active step, this prevents a spurious $\gamma V(s_T)$ bootstrap being added on top of the terminal outcome for time-capped games (the prior code set `done` only on death).

PopArt value normalization. The MSE critic target spans a huge dynamic range (the $\pm 1000/\sigma_R$ terminal spike vs. $O(10^{-2})$ per-step shaping). v6 (`USE_POPART`) trains the value head in a *normalized* space with output-preserving rescaling (Hessel et al., 2018). With running statistics over GAE returns $\hat{R}$, EMA rate $\beta = \text{POPART_BETA} = 3 \times 10^{-4}$,

$$\mu \leftarrow (1-\beta)\mu + \beta \mathbb{E}[\hat{R}], \quad \nu \leftarrow (1-\beta)\nu + \beta \mathbb{E}[\hat{R}^2], \quad \sigma = \sqrt{\nu - \mu^2}, \quad (13)$$

the value loss regresses the normalized target, $\mathcal{L}^V = (\tilde{V}_\theta(s_t) - (\hat{R}_t - \mu)/\sigma)^2$, while GAE/bootstrapping use the de-normalized $V = \sigma \tilde{V}_\theta + \mu$. On each statistics update $(\mu, \sigma) \rightarrow (\mu', \sigma')$ the final value layer is rescaled to *preserve* raw outputs,

$$W \leftarrow W \frac{\sigma}{\sigma'}, \quad b \leftarrow \frac{\sigma b + \mu - \mu'}{\sigma'}, \quad (14)$$

so $V(s) = \sigma \tilde{V}_\theta(s) + \mu$ is invariant to the update (verified to 4×10^{-7}). PopArt state is persisted in the resumable train-state.

Everything else. The clipped surrogate (`rl_math.tex eq:clip`), the log-ratio clamp, `vf_coef`, `MINIBATCHES`, `MAX_GRAD_NORM`, and the KL definitions are unchanged (see `rl_math.tex` § “**KL / trust region**” (`sec:kldiv`)). `kl_beta=0` (no reference-policy anchor) for the from-scratch run.

7 Systems / data path (A100, compute-bound)

The simulator *dynamics* are byte-identical to see `rl_math.tex` § “**Batched GPU simulator**” (`sec:gpuenv`); v6 only changes how the rollout and update move data so the GPU is the bottleneck (no math change):

- **bf16 autocast** on every net forward (rollout *and* update); the distribution log-prob / entropy / value and the GAE remain fp32. bf16 has fp32 range, so **no GradScaler** — safe against the Dirichlet `log_prob`.
- **On-GPU rollout buffers + GPU GAE**: the per-step (ent, mask, act, $\log \pi$, V , r , done) buffers live on-device and the GAE recursion runs on-GPU — no host copies, no `.cpu()/item()/synchronize` in the step loop. The only removed CPU sync (the per-step comet-schedule `.item()` and the early-break) is replaced by passing the Python step index.
- **Flash SDPA** attention kernel (§2a) and **fused Adam**.
- Minibatch tensors are indexed *on-device* in the update (no per-minibatch host→device copy).
- Batch $B = 256$ (`NUM_GROUPS`×`GROUP_SIZE` = 16×16) for an A100; memory is dominated by the rollout buffers (≈ 4 GB at $B=256, T=500$), comfortably within a 40/80 GB A100.

8 Config ↔ symbols (delta only)

New or changed knobs (all in the v6 config cell); unchanged symbols are in see `rl_math.tex` § “**Symbols ↔ config**” (`(symbols table)`).

config	symbol / role	v6 value (vs. rl_math)
GAMMA	γ (GAE discount)	0.997 (was 0.99)
GAE_LAMBDA	λ (trace decay)	0.98 (was 0.95)
UPDATE_EPOCHS	PPO epochs/iter	3 (was 1)
KL_EARLYSTOP	end-of-epoch KL stop	on (was per-minibatch break)
USE_POPART, POPART_BETA	value normalization β	on, 3×10^{-4} (new)
ENT_COEF_START/END/DECAY_ITERS	c_H anneal	0.02 $\rightarrow$ 0.002 / 300 (was fixed)
USE_POTENTIAL_SHAPING	reward family	on (replaces C, D, P)
SHAPE_SHIP, SHAPE_PROD	$c_{\text{ship}}, c_{\text{prod}}$	50, 50 (new)
DEST_HEAD	WHERE head type	bilinear (was linear)
INIT_DEST_SCALE	calm-init WHERE scale	0.02 (new)
GREEDY_TAU	τ , sharpened greedy temp	0.5 (new; was Dirichlet mean)
GATE_TRIM_LO/HI, GATE_EPS, INIT_GATE_BIAS	gate p_s trim/init	0.2, 0.8, 0.02, -1 (gate is new vs. rl_math)
USE_AMP, AMP_DTYPE	bf16 autocast	on, bf16 (new)
B	parallel envs	256 (was 128)

9 Appendix: model scaling (transformer)

Since v6 runs no MCTS, the net *is* the agent and the binding constraint on an A100 is wall-clock $\times$ iterations, not memory. A scaling check is a *ladder* of runs differing only in the transformer stack, all else fixed (seed, reward, league, B , T), compared on the gauntlet score of §5 vs. both iteration and wall-clock. With per-layer parameter count $\approx 12d^2$ and per-iter forward cost $\propto (12N_{\text{tx}} + c)d^2$ (dominated by the T sequential rollout forwards), a geometric ladder is e.g.

S ($d=256, N_{\text{tx}}=4$) $\rightarrow$ M (384, 6) $\rightarrow$ L (512, 8) $\rightarrow$ XL (768, 12), $\approx 4\text{M} \rightarrow 12\text{M} \rightarrow 28\text{M} \rightarrow 90\text{M}$ params,

with $N_{\text{HEADS}} = d/64$. RL scaling is not guaranteed monotonic (bigger nets are more sample-hungry and less stable; use LR warmup and lower LR for L/XL); read the gauntlet-vs-wall-clock curve for the compute-optimal size, and keep the deployable size within the Kaggle CPU turn budget.